SYSTEM-LEVEL SOFTWARE DESIGN SPECIFICATION

TikTockIT Ticketing Application

CPE 334 Software Engineering Course Project

Document status: Draft v0.9 for design review. Recommended defaults are complete enough to guide feature specifications, but the decisions in the Decision Register require confirmation.

Document Control	Value
Document ID	SDS-SYS-001
Version	0.9 Draft
Prepared	17 August 2026
Method	Spec-Driven Development (SDD)
Design level	System-wide architecture and cross-cutting standards
Feature designs	Maintained separately and required to reference this SDS

Design objective

Provide one authoritative system design baseline so that students, instructors, and coding agents implement features consistently without repeatedly deciding architecture, security, data, API, UI, testing, and deployment conventions.

Decision Register

The recommended defaults below are used throughout this draft. Confirming them converts the document from Draft to Approved. A changed decision must be reflected here and in all affected feature specifications.

ID	Decision	Recommended default	Confirmation needed
D-01	Official product spelling	TikTockIT	Confirm because prior materials also contain TokTickIT.
D-02	Ticket status vocabulary	New, Assigned, In Progress, Pending Requester, Resolved, Closed, Cancelled	Confirm statuses plus reopen and cancellation rules.
D-03	Priority vocabulary	Low, Medium, High, Urgent	Applies to Requested Priority and IT Priority.
D-04	Authentication session design	Opaque server-side session stored in PostgreSQL; Secure, HttpOnly, SameSite=Lax cookie	Recommended for course simplicity and safer browser storage.
D-05	Account provisioning and password reset	Administrator creates accounts; one-time email reset added only if notifications are included	Choose admin-created, self-registration, or institutional identity.
D-06	Attachment storage	Google Cloud Storage in production; storage adapter with local emulator/test implementation	Database stores metadata, never file bytes.
D-07	Notification channels	In-app indicators for MVP; email is optional integration	Confirm whether email delivery is required in the graded scope.
D-08	GCP deployment topology	One Cloud Run service serves SPA and API; Cloud SQL for PostgreSQL; Cloud Storage for files	Logical tiers remain separate even with one application container.
D-09	Branding and theme	Use the provisional accessible palette in this SDS	Replace with official university or project brand rules if supplied.
D-10	Ticket number	TKT-YYYY-NNNNN, generated transactionally	Confirm prefix and annual sequence reset.
D-11	Retention and recovery	No ticket hard deletion; daily database backups retained 7 days; deleted attachment binary removed immediately	Confirm institutional retention and recovery targets.
D-12	Runtime and library versions	Use course-approved current stable/LTS majors; pin exact versions and lockfile before Sprint 1	Exact version matrix should be frozen in the repository.

MY ANSWERS:

Here are my answers to your question:

D-01: Product name is TokTickIT

D-02: Ticket status looks good. Let anyone cancel whenever. Let anyone reopen whenever.

D-03. Priority looks good.

D-04. As you recommend.

D-05. Admin creates account and password. No email integration. Assume if is conveyed manually.

D-06 - No Google cloud for labs. Just think of a simple open-source object storage system that stores on the server. We don't expect that much storage space needed.

D-07 - In-app indicator is good.

D-08 - All locally deployed on one Server. Keep it simple.

D-09 - Use KMUTT university color palette.

D-10. As per your recommendation.

D-11. As per your recommendation.

D-12. As per your recommendation.

Document Map

- Purpose, scope, assumptions, and design principles
- Architecture, technology stack, deployable structure, and shared components
- Data architecture, security, roles, workflow invariants, APIs, and UI standards
- Attachments, auditability, NFR realization, deployment, testing, and feature-specification governance
- References and glossary

Purpose and Scope

Purpose

This System-Level SDS defines the design decisions that apply across every TikTockIT feature. It is the stable design baseline between the Software Requirements Specification (SRS) and the feature-level

design specifications. It defines how the system is structured and how shared concerns are handled; it does not replace detailed feature behavior, screens, APIs, acceptance criteria, or test cases.

Intended Audience

- Students implementing the application in a team using GitHub and coding agents
- Instructors and teaching assistants reviewing design consistency and engineering deliverables
- Developers authoring feature specifications, database migrations, APIs, UI components, and tests
- Reviewers verifying traceability from SRS requirements to implemented and tested behavior

System Scope

The design supports an internal IT service ticketing application with these system-wide capability groups:

- Email/password authentication and role-based access control
- Ticket creation, categorization, priority, ownership, status workflow, resolution, and closure
- Service Actions with assignees and a lifecycle separate from the Ticket Owner
- Attachments with controlled upload and audited removal
- Immutable ticket event history for material actions and changes
- Search, filtering, dashboards, and administrative reference data
- Automated tests, CI/CD, and deployment to Google Cloud Platform

Out of Scope for the System-Level SDS

- Complete page wireframes and feature-specific interaction sequences
- Exact endpoint payloads for every feature
- Sprint backlog, effort estimates, and grading rubrics
- External SLA, procurement, CMDB, asset management, chat, or AI support capabilities unless added to the SRS

Design Basis

This draft preserves the following course and application decisions:

- Frontend: React, TypeScript, Vite, and Bootstrap
- Backend: Node.js, Express, and TypeScript
- Database: PostgreSQL with Prisma ORM and migrations
- Architecture style: REST-style APIs and logical three-tier architecture
- Roles: Requester, IT Staff, and Administrator. The role name Agent is not used.

- Testing: Vitest, Supertest, and Playwright
- Workflow: Git, GitHub Issues/Projects, pull requests, and GitHub Actions
- Deployment target: Google Cloud Platform

Design Principles

1. Trace every system behavior to an approved requirement, business rule, NFR, or design decision.
2. Keep business rules in backend domain services so UI code cannot bypass them.
3. Use database transactions for state changes that must be accompanied by an event record.
4. Apply least privilege and deny access by default.
5. Prefer one clear, course-appropriate implementation over unnecessary infrastructure complexity.
6. Design for automated testing at unit, API integration, and browser end-to-end levels.
7. Keep the system-level rules stable; feature specifications may extend but may not contradict them.

System Architecture

Architecture Style

TikTockIT uses a logical three-tier architecture. The presentation tier contains the React SPA. The application tier contains Express routing, authentication, authorization, validation, and domain services. The data tier contains PostgreSQL and object storage. Deployment may package the first two tiers into one Cloud Run container without merging their code responsibilities.

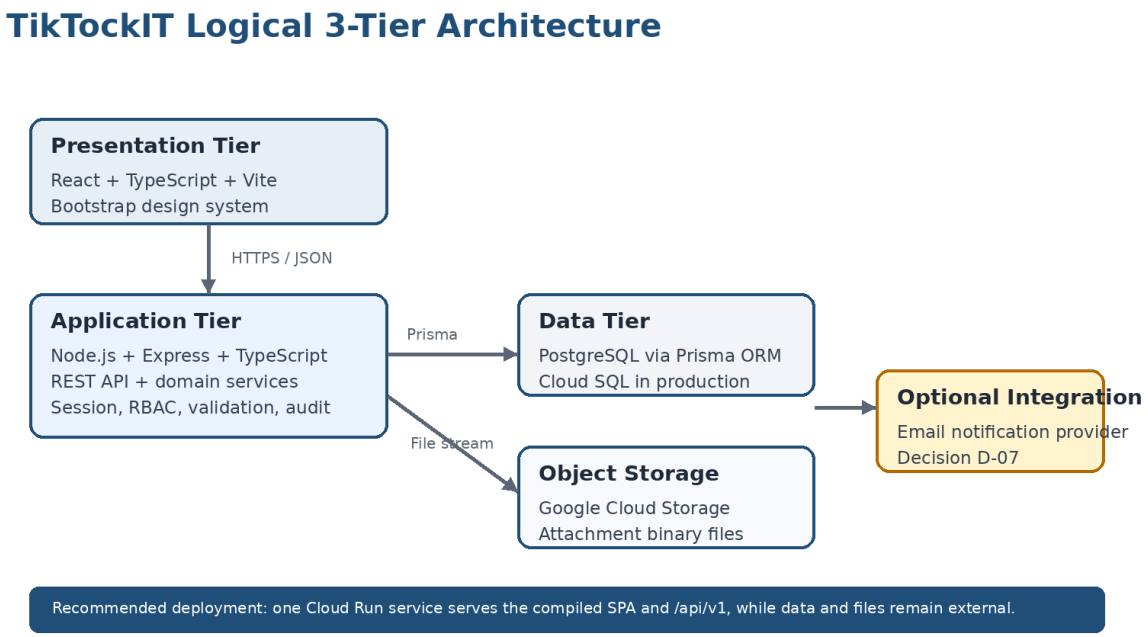

Figure 1. Recommended logical and production architecture

Decision D-08: Confirm the recommended single Cloud Run application service. A later course iteration may split the SPA and API into separate deployables without changing the logical architecture.

Deployment Units

Deployable/Service	Recommended implementation	Responsibility
Web application	Compiled React SPA served as static assets by Express	Browser presentation tier
Application API	Express routes under /api/v1 plus domain services	Application tier
Relational database	PostgreSQL in Cloud SQL	Transactions, constraints, system records
Object storage	Google Cloud Storage	Attachment binaries; metadata remains in PostgreSQL
CI/CD	GitHub Actions	Build, test, scan, migrate, and deploy

Technology Stack

Layer	Technology	System-level use
Language	TypeScript	Shared type discipline; no direct sharing of persistence models with public API contracts
Frontend	React + Vite + Bootstrap	Component UI, routing, forms, accessible responsive layout
Backend	Node.js + Express	REST API, sessions, RBAC, validation, orchestration
Persistence	PostgreSQL + Prisma	Relational integrity, migrations, transactions, typed data access
Unit/API testing	Vitest + Supertest	Domain and endpoint verification
E2E testing	Playwright	User-visible workflows in a real browser
Delivery	GitHub Actions + GCP	Repeatable test and deployment pipeline

Repository Structure

```
client/              React application and UI components
server/              Express API, middleware, domain services
server/prisma/       Prisma schema, migrations, and seed data
shared/              API contract types and shared constants only
tests/e2e/           Playwright tests
.github/workflows/   CI/CD pipelines
docs/specs/          SRS, System-Level SDS, and feature specifications
```

Shared Components and Boundaries

Component	Responsibility	Boundary rule

Component	Responsibility	Boundary rule
Web UI	Routes, forms, tables, dashboards, accessible feedback	Calls only published API contracts
API Controller	HTTP parsing, status codes, DTO mapping	Contains no core business rules
Authentication	Login, logout, session validation, password verification	Produces authenticated user context
Authorization	Role and resource ownership checks	Enforced server-side on every protected operation
Ticket Domain Service	Ticket lifecycle, owner, priorities, resolution rules	Owns ticket invariants and transactions
Service Action Service	Action lifecycle and assignee rules	Enforces action state transitions
Attachment Service	Validation, object storage, audited removal	Never exposes raw storage keys to clients
Audit/Event Service	Append-only event creation and retrieval	Events created in the same transaction as changes
Repositories	Prisma queries and persistence mapping	No HTTP concerns

Domain Data Model

All primary entities use UUID identifiers. Human-readable ticket numbers are separate unique values. Timestamps are stored in UTC using PostgreSQL timestamptz. Prisma enums express stable status and priority vocabularies after D-02 and D-03 are confirmed.

Entity	Important fields	System-level rule
User	id, email, passwordHash, displayName, role, isActive, createdAt, updatedAt	Email unique; role is Requester, IT Staff, or Administrator
Ticket	id, ticketNo, title, description, requesterId, ownerId, categoryId, requestedPriority, itPriority, status, resolutionSummary, requesterResolutionConfirmedAt, version, timestamps	Requester required; owner nullable until assignment; optimistic version increments on updates
Category	id, code, name, description, isActive	Inactive categories remain referenced by historical tickets
ServiceAction	id, ticketId, title, description, assigneeId, status, dueAt, completionNote, timestamps	Status is Planned, In Progress, Completed, or Cancelled
Attachment	id, ticketId, uploadedById, originalFilename, mimeType, sizeBytes, storageKey, deletedAt, deletedById	Binary stored externally; deleted row becomes a tombstone
TicketEvent	id, ticketId, actorId, eventType, payloadJson, createdAt	Append-only; payload stores audit detail appropriate to event type
Session	id, userId, expiresAt, revokedAt, metadata	Required only if D-04 server-side sessions are approved

Relationship and Integrity Rules

- A User may request many Tickets and may own many Tickets if the user is IT Staff or Administrator.
- A Ticket has zero or more Service Actions, Attachments, and Ticket Events.
- A Service Action assignee may differ from the Ticket Owner.
- Tickets and Ticket Events are never cascade-deleted. Foreign-key deletion is restricted for historical records.
- Users and Categories use activation flags rather than hard deletion when referenced.
- Ticket creation, ticket transitions, priority changes, ownership changes, and attachment removal each write an event in the same database transaction as the business change.

Concurrency and Transactions

- Ticket updates use an integer version field. A stale update returns HTTP 409 Conflict rather than overwriting newer work.
- Ticket numbers are generated transactionally so concurrent creates cannot produce duplicates.
- Multi-record business changes use Prisma transactions.
- Object storage operations use compensating cleanup when a database transaction cannot include the external file operation.

Security Architecture

Authentication

Users authenticate with email and password. Passwords are never stored or logged in plaintext. The recommended algorithm is Argon2id using a maintained Node.js library and parameters benchmarked for the deployment environment, following OWASP guidance [R1].

- Email comparison is case-insensitive and normalized before uniqueness checks.
- Login responses do not reveal whether an email address exists.
- Repeated failed logins are rate-limited and security-relevant failures are logged without credentials.
- Passwords are accepted as Unicode and are never silently truncated.
- Password reset tokens, if implemented, are single-use, time-limited, and stored as hashes.

Decision D-04 and D-05: Approve the server-side session design and account-provisioning model before the authentication feature specification is written.

Session Management

The recommended design stores an opaque session identifier in a Secure, HttpOnly, SameSite=Lax cookie and stores session state in PostgreSQL. Session identifiers rotate after login and privilege changes. Logout and password reset revoke active sessions. Cookie flags and session expiration follow OWASP session-management guidance [R2].

- Idle timeout: 30 minutes. Absolute timeout: 8 hours. Both are provisional security targets.
- State-changing requests require CSRF protection when cookie authentication is used.
- The production application is HTTPS-only and sets HSTS at the edge.
- Client JavaScript never reads the session token.

Authorization Model

Authorization combines role checks with resource-level rules. The backend is authoritative; hiding a UI control is not authorization.

Capability	Requester	IT Staff	Administrator
Create a ticket	Yes	Yes	Yes
View tickets	Own/requested tickets	All tickets	All tickets
Set or change IT Priority	No	Yes	Yes
Change formal ticket status	No	Yes	Yes
Assign Ticket Owner	No	Yes	Yes
Create/manage Service Actions	No	Yes	Yes
Delete own attachment before Closed	Yes	Yes	Yes
Delete another user's attachment	No	Yes, with reason	Yes, with reason
Manage users, roles, categories	No	No	Yes
View security/administrative audit	No	Restricted	Yes

Input and Output Security

- Validate all request payloads at the API boundary and enforce database constraints independently.
- Use Prisma parameterization; never construct SQL with untrusted string concatenation.
- React renders user content as text by default. Raw HTML rendering is prohibited unless sanitized by an approved library.
- Production errors return safe messages and correlation IDs, not stack traces or database details.
- Secrets are injected from environment configuration or Google Secret Manager and never committed to Git.

Cross-Feature Workflow Rules

Ticket Status Baseline

Status	Meaning	Invariant
New	Created and not yet formally assigned	Owner may be empty
Assigned	Ticket Owner has accepted responsibility	Owner required
In Progress	Work is actively underway	Owner required
Pending Requester	IT needs requester input or verification	Owner remains responsible
Resolved	IT Staff has formally resolved the ticket	All Service Actions completed or cancelled
Closed	Final terminal state	No normal modification; attachment upload/removal disabled
Cancelled	Ticket will not be completed	Reason required; terminal unless Administrator reopens

Decision D-02: Confirm the status list, whether Resolved tickets can be reopened, who may cancel, and whether Cancelled is terminal. Until confirmed, feature specifications must label these transitions provisional.

Mandatory Ticket Invariants

- Only IT Staff or Administrator changes the formal ticket status to Resolved or Closed.
- The Requester may indicate that the problem appears resolved or confirm resolution, but this records a confirmation flag/event and does not directly change formal status.
- If requester confirmation exists while open Service Actions remain, the Ticket Owner is warned and the ticket stays in its current status.
- A ticket cannot move to Resolved or Closed while any Service Action is Planned or In Progress.
- Before resolution, every Service Action must be Completed or Cancelled.
- Ticket Owner responsibility is separate from Service Action assignment.
- Requested Priority belongs to the Requester. IT Priority is controlled by IT Staff or Administrator. IT Priority may initially copy Requested Priority.
- All changes to IT Priority are recorded in the Ticket Event log.

Service Action Lifecycle

Status	Meaning	Allowed next states

Status	Meaning	Allowed next states
Planned	Defined but not started	In Progress, Cancelled
In Progress	Actively being performed	Completed, Cancelled
Completed	Finished with completion evidence	Terminal
Cancelled	No longer required; reason recorded	Terminal

API Design Standards

- All application endpoints are rooted at /api/v1.
- Requests and responses use JSON except multipart attachment upload and file download.
- Property names use camelCase. Database naming may use snake_case through Prisma mapping.
- Dates and times use ISO 8601 UTC strings. Display localization occurs in the client.
- Collection endpoints support pagination, explicit filtering, and a constrained sort whitelist.
- Successful create operations return HTTP 201; validation errors 422; authentication failures 401; authorization failures 403; missing resources 404; stale updates 409.
- API contracts use DTOs. Prisma models are never serialized directly.

```
{
  "error": {
    "code": "TICKET_STATE_CONFLICT",
    "message": "Ticket cannot be resolved while service actions remain open.",
    "fieldErrors": [],
    "correlationId": "..."
  }
}
```

Validation and Error Handling

- Client validation improves usability but never substitutes for API validation.
- Validation schemas are reusable at API boundaries and return stable machine-readable error codes.
- Domain-rule failures are distinct from input-format failures.
- Unexpected exceptions are logged with a correlation ID and mapped to a generic HTTP 500 response.
- The UI preserves user input after recoverable errors and clearly identifies fields requiring correction.

Frontend and UI Design System

Frontend Structure

- Route components orchestrate feature pages; reusable components remain presentation-focused.
- API access is centralized in typed service modules rather than called directly from arbitrary components.
- Authentication state and current-user role are exposed through one application-level context.
- Server state is refreshed after mutations; the client does not assume a status change succeeded.
- Permission-sensitive controls are hidden or disabled for usability, while the server still enforces permission.

Provisional Theme Tokens

Token	Value	Use
Primary	#1F4E78	Navigation, primary actions, key headings
Action blue	#2E75B6	Links, focus accents, information
Text	#1F2937	Primary text
Muted	#5B6573	Secondary labels and metadata
Background	#F7F9FC	Application background
Success	#2E7D32	Completed/success state, always with text/icon
Warning	#B26A00	Attention state, always with text/icon
Danger	#B3261E	Destructive/error state, always with text/icon

Decision D-09: Confirm whether TikTockIT should use this neutral palette, university branding, or another corporate style guide. The SRS should state the branding constraint; this SDS records the exact tokens.

System-Wide UI Standards

- Use the system font stack with Bootstrap's responsive grid and spacing scale.
- Use one shared component treatment for forms, buttons, tables, badges, confirmation dialogs, loading, empty, and error states.
- Do not communicate status or priority by color alone. Include text and, where useful, an icon.
- Every form control has a programmatic label; validation messages are associated with the field.
- Keyboard focus is visible and logical. Dialogs trap focus and return it to the invoking control.

- Target WCAG 2.2 Level AA [R4]. Feature specifications must identify any justified exception.
- Use Bootstrap breakpoints unless a feature specification documents a stronger layout need.
- Dates display in the user's locale but are transported and stored in UTC.

Attachment Architecture

Both Requesters and IT Staff may add attachments when ticket permissions and status permit. The uploader may delete their own attachment while the ticket is not Closed. IT Staff or Administrators may remove an attachment when necessary. Every removal requires confirmation and creates an ATTACHMENT_REMOVED event with filename, uploader, remover, reason, and timestamp.

Upload Controls

- Suggested limit: 5 MB per file and no more than 5 active files per ticket. Confirm these values in D-06 or the SRS.
- Allowed extensions and MIME types: JPG, PNG, WEBP, and PDF only.
- Validate extension, detected content type, declared MIME type, size, and authorization. Rename stored objects to generated identifiers.
- Do not execute, render as HTML, or serve attachments inline with an attacker-controlled content type.
- Use authenticated download endpoints or short-lived signed URLs after an authorization check.
- Apply layered upload controls following OWASP file-upload guidance [R3].

Removal and Audit Sequence

1. Authorize the remover and verify the ticket is not Closed unless an Administrator override is permitted.
2. Require confirmation and a removal reason.
3. Mark the Attachment metadata as deleted and create ATTACHMENT_REMOVED in one database transaction.
4. Delete the object from storage. If deletion fails, queue or record a retry without restoring user visibility.
5. Display the removal event in the authorized ticket history while never exposing the deleted object key.

Event and Audit Model

- Ticket Events are append-only and are never edited through normal application APIs.

- Events record actor, event type, timestamp, and the minimum structured before/after detail needed for accountability.
- Passwords, session IDs, reset tokens, attachment content, and unrelated personal data never enter event payloads.
- Material events include ticket creation, ownership change, status change, IT Priority change, requester resolution confirmation, Service Action lifecycle change, attachment addition/removal, and administrative changes.
- Operational logs and business events are separate. Operational logs support diagnosis; Ticket Events support domain history.

Non-Functional Design Realization

The SRS owns the authoritative NFR values. The targets below are provisional engineering baselines so feature designs are not ambiguous. Replace any target that conflicts with an approved NFR.

Quality	Target	Design response
Performance	p95 API response under 500 ms for normal CRUD at 50 concurrent users; uploads excluded	Indexes, pagination, query review, payload limits, performance smoke test
Availability	Best effort course service; no contractual SLA	Health endpoint, stateless app process, managed database
Scalability	Horizontal application scaling without local session or file dependence	PostgreSQL session store, Cloud Storage files, Cloud Run
Security	Least privilege, secure password/session handling, dependency review	RBAC, CSRF, rate limiting, secrets management, CI checks
Accessibility	WCAG 2.2 AA target	Semantic HTML, keyboard support, contrast, Playwright/a11y checks
Maintainability	Clear layer boundaries and typed contracts	Lint, formatting, code review, migrations, domain services
Auditability	All material ticket changes traceable	Append-only TicketEvent written transactionally
Recoverability	Daily backups retained 7 days; provisional	Cloud SQL backups and documented restore exercise
Privacy	Collect only data needed for service delivery	Access control, log redaction, retention decision, no hard delete of audit history

Observability

- Write structured JSON logs in production with timestamp, severity, correlationId, route, outcome, and safe user/ticket identifiers where appropriate.
- Expose /health for process health and a protected readiness check for database connectivity.
- Record latency and error counts by route without logging request bodies by default.

- Use one correlation ID across HTTP response, operational logs, and background work.
- Alerting thresholds and notification recipients are deployment-environment configuration, not code constants.

Configuration and Secrets

- Maintain separate development, test, staging, and production configuration.
- Validate required environment variables at startup and fail fast with a safe message.
- Store production secrets in Google Secret Manager or protected service configuration.
- Commit an .env.example containing names and safe examples only.
- Never use production credentials or production data in automated tests.

CI/CD and Deployment

Every pull request and protected-branch deployment follows a reproducible pipeline.

1. Install from the committed lockfile.
2. Run formatting and lint checks.
3. Run TypeScript compilation/type checking.
4. Run unit and API integration tests against an isolated PostgreSQL database.
5. Build the React application and application container.
6. Run Playwright smoke tests in an environment suitable for browser execution.
7. Apply reviewed Prisma migrations using a controlled migration job before or during deployment.
8. Deploy to staging, run smoke tests, then promote the same immutable artifact to production.

Database Migration Rules

- Every schema change is represented by a committed Prisma migration.
- Migrations are reviewed with the feature pull request and tested from a clean database plus the previous baseline.
- Destructive migration requires an explicit data-migration or rollback plan.
- Application and migration order must preserve backward compatibility during deployment where practical.
- Production migration credentials are separate from ordinary application credentials when feasible.

Testing Architecture

Level	Tool	Primary coverage	Boundary

Level	Tool	Primary coverage	Boundary
Unit	Vitest	Pure domain rules, validators, mappers, utilities	Fast; no network or real database
API integration	Vitest + Supertest	Routes, middleware, RBAC, transactions, Prisma repositories	Isolated PostgreSQL test database
Browser E2E	Playwright	Critical Requester, IT Staff, and Administrator workflows	Runs against deployed or local full stack
Migration	Prisma + test database	Clean install and upgrade from prior schema	Required for schema-changing PRs
Security	Automated checks + focused tests	Authorization regression, upload rejection, session/CSRF behavior	Required for protected operations

Mandatory Business-Rule Tests

- Requester cannot directly set Resolved or Closed.
- Open Service Actions prevent resolution and closure.
- Requester resolution confirmation is recorded without bypassing open actions.
- Only IT Staff or Administrator changes IT Priority after creation.
- Ticket Owner and Service Action Assignee may differ.
- Attachment removal permissions, ticket-state restriction, confirmation, reason, binary deletion, and event creation behave as specified.
- All protected endpoints reject unauthorized access even when called outside the UI.

Line coverage is a diagnostic metric, not a substitute for these behavior tests. A feature is not complete when its critical business rules are untested.

Feature Specification Contract

Each feature-level design specification inherits this System-Level SDS. A feature specification documents only its feature-specific behavior and design choices, while referencing rather than duplicating shared rules.

Section	Required content
Identity	Feature ID, name, purpose, owner, version, status
Traceability	Related BR, FR, NFR, business-rule, and decision IDs
Behavior	Actors, preconditions, main flow, alternatives, errors, acceptance criteria
Permissions	Role and resource-level access, including denial cases
Workflow	State transitions and system invariants affected

Section	Required content
Data	Entities/fields changed, migration, constraints, transaction boundaries
API	Endpoints, DTOs, status codes, errors, pagination/filtering where relevant
UI	Routes, components, states, validation, accessibility, responsive behavior
NFRs	Applicable system targets plus feature-specific targets
Testing	Unit, integration, E2E, security, and acceptance test obligations
Dependencies	Other features, reference data, integrations, configuration
Out of scope	Explicit exclusions to prevent coding-agent assumptions

Specification Precedence

1. Approved SRS requirements and business rules define what must be achieved.
2. This System-Level SDS defines shared design decisions and constraints.
3. An approved feature specification defines feature-specific behavior and design.
4. Implementation tasks divide the approved feature specification into work items.
5. Automated and acceptance tests verify the specifications.

> **Conflict rule:** A feature specification may extend this SDS but may not silently contradict it. Resolve conflicts through an approved SDS decision change and update traceability before implementation.

Definition of Ready for Implementation

- The feature maps to approved requirements and has no unresolved business ambiguity.
- Applicable system-level decisions are approved or explicitly accepted as provisional for the sprint.
- Behavior, permissions, data, API, UI states, error handling, and acceptance criteria are defined.
- Dependencies and migration implications are understood.
- The test obligations are concrete enough to write before or alongside implementation.
- A developer or coding agent can implement without inventing important business rules.

Traceability and Change Control

- Maintain a matrix from SRS requirement IDs to feature IDs, feature specification versions, pull requests, and test IDs.
- Record system-level changes in this document's Decision Register and version history.

- Do not alter an approved shared enum, role, workflow invariant, API convention, or data-lifecycle rule in one feature only.
- Pull requests reference the implementing issue and feature specification. Issue and pull-request numbers are independent identifiers.
- Architecture-significant changes require instructor or designated design-review approval before merge.

System-Level Review Checklist

Review area	Pass condition
Architecture	Layer boundaries and deployable topology remain consistent
Security	Authentication, authorization, session, upload, and secret rules are followed
Data	Constraints, migrations, transactions, audit events, and deletion behavior are defined
API	Versioning, DTOs, status codes, validation, and error envelope are followed
UI	Shared theme, components, responsive behavior, and accessibility are followed
Testing	Critical business rules are verified at the appropriate test level
Traceability	Requirement, feature, issue, PR, and test links are present

References

ID	Reference	URL
R1	OWASP Password Storage Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
R2	OWASP Session Management Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
R3	OWASP File Upload Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
R4	W3C Web Content Accessibility Guidelines (WCAG) 2.2	https://www.w3.org/TR/WCAG22/
R5	Google Cloud Run Container Runtime Contract	https://cloud.google.com/run/docs/container-contract
R6	Google Cloud Storage Objects Overview	https://cloud.google.com/storage/docs/objects

Glossary

Term	Meaning
BR	Business Requirement
FR	Functional Requirement
NFR	Non-Functional Requirement
SRS	Software Requirements Specification
SDS	Software Design Specification
SDD	Spec-Driven Development in this course
Requester	User who requests IT support and may confirm that a problem appears resolved
IT Staff	Role responsible for formal ticket workflow and service work
Administrator	Role with IT Staff authority plus system administration
Ticket Owner	IT Staff or Administrator accountable for the ticket as a whole
Service Action Assignee	Person assigned to an individual Service Action; may differ from Ticket Owner
Ticket Event	Append-only business audit record associated with a ticket

Approval Gate

This draft becomes the approved System-Level SDS after the Decision Register is confirmed and the authoritative SRS/Feature Inventory traceability matrix is attached or referenced. Feature specifications may be drafted earlier, but must identify any dependency on an unresolved decision.

Next action: Confirm D-01 through D-12. I recommend resolving D-01 to D-05 first because naming, workflow, priorities, authentication, and account provisioning affect the earliest feature specifications.